

Proyecto 3

1 Descripción del proyecto

El proyecto es **individual**. Consiste en extender la aplicación web del Proyecto 2 —gestión de inventario y ventas de una tienda— para incorporar seguridad a nivel de base de datos mediante un esquema de roles y permisos, stored procedures y el uso obligatorio de un ORM.

Todo lo implementado en el Proyecto 2 debe estar presente y funcional. Es un requisito de entrega, no un criterio de calificación: si la aplicación base no funciona, no se evalúa este proyecto.

Entrega

El trabajo se entrega en el **mismo repositorio de GitHub del Proyecto 2**, en una rama llamada `proyecto-3`. No se evaluará código fuera de esa rama. El README debe actualizarse para reflejar los cambios e incluir instrucciones para levantar el proyecto desde cero con `docker compose up`.

Contexto del negocio

La tienda maneja productos agrupados en categorías, comprados a proveedores. Los clientes realizan compras atendidas por empleados. Cada compra puede incluir varios productos y debe quedar registrada junto con el detalle de lo vendido. La tienda necesita controlar el stock disponible, generar reportes de ventas y garantizar que cada tipo de usuario solo pueda realizar las operaciones que le corresponden.

2 Restricciones técnicas

- **DBMS relacional:** el mismo utilizado en el Proyecto 2 (MySQL, PostgreSQL, SQLite o SQL Server).
- **ORM obligatorio.** A diferencia del Proyecto 2, el uso de un ORM es **requerido**. Las operaciones CRUD de la aplicación deben realizarse a través del ORM; las consultas avanzadas (subqueries, CTEs, reportes) pueden complementarse con SQL explícito.
- **Roles en la base de datos.** Deben definirse **exactamente 5 roles** en el DBMS mediante `CREATE ROLE` con permisos granulares asignados con `GRANT` y `REVOKE`. Los roles deben corresponder a tipos de usuario con responsabilidades distintas dentro del negocio.
- **Stored procedures obligatorios.** Las operaciones críticas del negocio deben implementarse como stored procedures invocados desde el backend.
- **Docker obligatorio.** Toda la infraestructura debe definirse en `docker-compose.yml` y levantarse con `docker compose up`.

- **Variables de entorno:** credenciales en `.env`; el repositorio incluye `.env.example`.
- **Credenciales fijas para calificación:** usuario `proy3`, contraseña `secret`. No se calificará un proyecto con credenciales distintas.
- Las transacciones deben marcarse explícitamente (`BEGIN` / `COMMIT` / `ROLLBACK` o equivalente del ORM).
- Rama de entrega: `proyecto-3` en el repositorio del Proyecto 2.

3 Rúbrica de evaluación

Cada tarea se califica como aprobada o no aprobada; no hay puntaje parcial por tarea. **Todos los criterios son obligatorios.** La nota máxima es **100 puntos**.

I. Seguridad y roles

Los roles deben existir en el DBMS, no solo como lógica de aplicación.

5 roles definidos en el DBMS con CREATE ROLE y permisos granulares por tabla u operación mediante GRANT y REVOKE	20
Esquema de roles documentado: nombre de cada rol, tablas accesibles y operaciones permitidas	10
Autenticación con sesión (login/logout) y un usuario de prueba funcional por cada rol incluido en el script de datos	10
Rutas y vistas de la UI protegidas según el rol del usuario autenticado	15

Subtotal **55**

II. Stored Procedures y ORM

Al menos 5 stored procedures invocados desde el backend (no desde scripts independientes)	15
Al menos 1 stored procedure con parámetros de entrada/salida y manejo de excepciones	10
Al menos 1 transacción explícita con ROLLBACK implementada dentro de un stored procedure	10
ORM configurado y utilizado en al menos 3 operaciones CRUD de la aplicación	10

Subtotal **45**

Total 100

Notas:

- El trabajo debe entregarse en la rama `proyecto-3` del repositorio del Proyecto 2. No se evaluará código en otra rama o repositorio.
- Una aplicación que no incluya todo lo requerido en el Proyecto 2 o que no levante con `docker compose up` no se evalúa.
- Un proyecto con credenciales distintas a `proy3 / secret` no se evalúa en ningún criterio que requiera conexión a la base de datos.

- Un proyecto sin roles definidos en el DBMS no obtiene puntos en la Categoría I, aunque la aplicación controle el acceso a nivel de código.
- Las entregas se realizan según las fechas publicadas en Canvas. Todo el trabajo debe estar en el repositorio de GitHub antes del cierre.